Eastern England SECURE DATA ENVIRONMENT | Part of the NHS Research Secure Data Environment Network

VISTA A DARE UK PROJECT

Eastern England SDE Researcher Model Disclosure Control Standard Operating Procedure

Table of Contents

1. Purpose 2
   1.1 Scope 3
   1.2 What SACRO-ML Contains 3
2. SafeModel (Safe package) 5
3. Attacks (Attack package) 5
   3.1 What are “simulated attacks” in SACRO-ML 6
   3.2 Types of Disclosure Risk Relevant to ML Models 6
   3.3 Attack types supported in SACRO-ML documentation 7
   3.4 Researchers should be aware of following supporting components 7
4. How SACRO-ML is used in the SDE workflow 7
   4.1 Responsibilities (researcher vs output checker) 7
   4.2 Typical outcomes 9
5. Researcher interaction with SACRO-ML (including examples) 9
   5.1 What is expected vs optional 9
   5.2 Example A: Using SafeModel during model development (illustrative workflow) 9
   5.3 Example B: Running a simulated attack as a researcher pre-check (optional) 10
6. SACRO-ML – Getting Started (Researchers) 10
   i. When Should You Use SafeModel? 10
   ii. Prerequisites 10
   iii. Standard Researcher Workflow with SafeModel 11
   iv. Decision Point 12
   v. Preparing for Disclosure Submission 12
   vi. Optional: Running Simulated Attacks (Advanced) 12
   vii. What Happens After Submission? 12
   viii. Full Documentation 13

Glossary 13

Author:
Kshitij Nemade (Eastern England Secure Data Environment, Health Innovation East, ORCID: 0009-0000-8670-0814)
Onesimus Ekechi (Eastern England Secure Data Environment, Health Innovation East, ORCID: 0009-0005-9040-0167)
Poojita Karchalkar (Eastern England Secure Data Environment, Health Innovation East, ORCID: 0009-0000-6973-4692)
Magali Ruffier (Eastern England Secure Data Environment, Health Innovation East, ORCID: 0000-0002-8386-1580)

1. Purpose

SACRO-ML is a set of tools for managing statistical disclosure control (SDC) risks associated with trained machine-learning (ML) models. It is intended for use in secure analytical settings such as Secure Data Environments (SDEs), where outputs must be assessed prior to release to reduce the risk of personal data disclosure.

SACRO-ML supports disclosure control for ML through two complementary approaches:

- **Ante-hoc assessment** (during model configuration/training decisions)
- **Post-hoc assessment** (after training, by testing the trained model under controlled simulated risk scenarios)

Why ML models require specific disclosure risk assessment

In ML, the released artefact is often not a table or statistic but a trained model (or pipeline) that can be queried repeatedly. Disclosure risk can arise because models may:

- Behave differently for records used in training versus unseen records (a common basis for membership inference risk)
- Leak sensitive information through confidence scores, probability outputs, or other rich model responses
- Present additional risk when trained on small or distinctive cohorts

SACRO-ML addresses this by combining:

- SafeModel approach that surfaces potential issues early (ante-hoc)
- An Attacks approach that estimates empirical privacy risk by testing model behaviour under controlled conditions (post-hoc)

1.1 Scope

This handbook applies when a project involves:

- training a ML model on sensitive data within the TRE/SDE
- considering release of a model artefact or model-derived artefacts outside the TRE/SDE

It is especially relevant where:

- the model is intended for external deployment, sharing, or reuse, or
- the model output (e.g., probability scores) could be queried repeatedly

1.2 What SACRO-ML Contains

SACRO-ML comprises two main functional components:

- **SafeModel (Safe package)** – supports ante-hoc risk assessment and structured packaging for review.
- **Attacks (Attack package)** – supports post-hoc evaluation via simulated attacks and standard reporting.

The SACRO-ML workflow is illustrated in Figure 1, with further guidance on its application provided below.

Eastern England SECURE DATA ENVIRONMENT | Part of the NHS Research Secure Data Environment Network

VISTA A DARE UK PROJECT

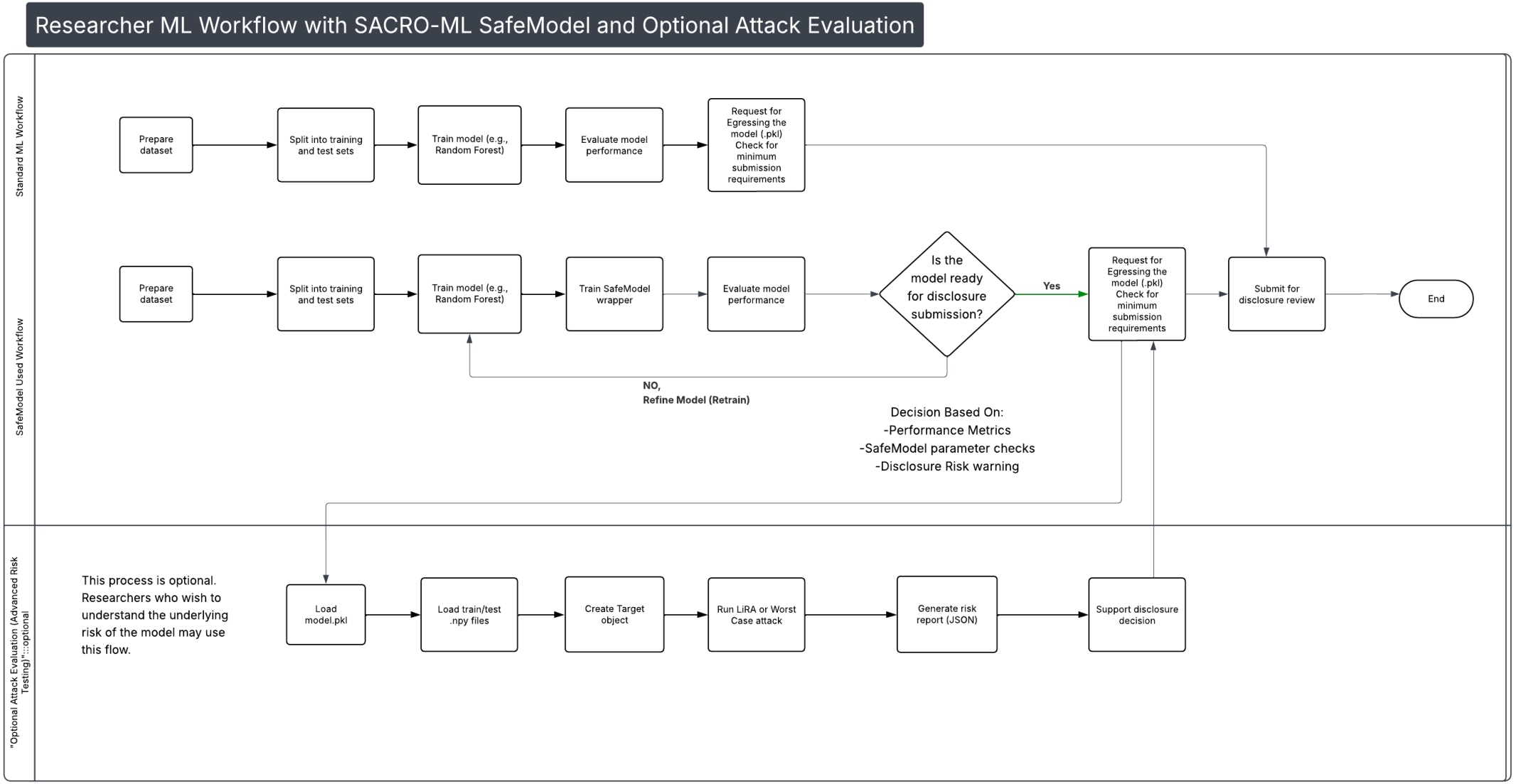


Figure 1: Depicts the Researcher workflow with and without the usage of SACRO-ML Tools.

2. SafeModel (Safe package)

SafeModel is an open-source wrapper approach for common machine-learning models designed for researchers working in TRE/SDE settings. It aims to increase confidence that model development and configuration choices are more aligned with disclosure control expectations by providing structured feedback.

SafeModel produces the “checkfile”

A core SafeModel output is a **JSON-parseable checkfile report** that records:

- researcher identifier (as configured)
- model type
- models save filename
- warnings/diagnostics about parameters that may present disclosure risk
- a recommendation indicating whether release should be allowed (in the tool’s logic)
- a reason/justification string for review

What SafeModel can do (capabilities relevant to researchers)

The SafeModel base class provides methods that support the safe release workflow, including:

- **preliminary_check**: checks whether model parameters violate “safe rules” and can optionally apply constraints
- **posthoc_check**: checks whether a model has been interfered with since training (fit)
- **save**: writes the model to file in an appropriate format
- **request_release**: saves model + creates a report for TRE output checkers

These capabilities define how SafeModel supports both:

- researcher self-checks (to reduce avoidable issues)
- structured submission artefacts for review

3. Attacks (Attack package)

Attack package is a collection of privacy risk assessment techniques that simulate membership inference and related attacks to test whether a trained model may leak information about the underlying training dataset.

3.1 What are “simulated attacks” in SACRO-ML

In SACRO-ML, simulated attacks are controlled scenarios in which a trained ML model is tested to estimate the risk of a data breach prior to model release. These tests are part of privacy risk assessment.

The SACRO-ML documentation highlights that attack simulation is relevant when:

- a model is trained using sensitive data and the objective is to reduce risk of leakage
- appropriate test data is available (i.e., data not seen during training), with practical guidance that test data should typically be a meaningful proportion of the dataset

It also notes limitations, including that attack simulation is not intended for anonymous/non-sensitive datasets and is not appropriate for certain model types that embed data (instance-based methods), and where no suitable test split exists.

3.2 Types of Disclosure Risk Relevant to ML Models

1. **Membership Inference Risk**

The risk that an attacker can determine whether a specific individual’s record was used in model training.

- Relevant when model overfits
- Measured via LiRA / Worst Case attacks
- Higher risk when train accuracy >> test accuracy

2. **Attribute Inference Risk**

The risk that an attacker can infer sensitive attributes of an individual using the model

- Relevant in models with highly predictive features
- Requires structured evaluation

3. **Structural Leakage**

Risk that the model structure itself leaks information (e.g. small leaves, memorisation in trees).

- Example: min_samples_leaf too small
- Controlled via SafeModel parameter checks

4. **Overfitting Risk**

When model generalisation error is high, increasing likelihood of memorisation.

- SafeModel flags parameter configurations
- Attack simulations quantify practical exploitability

3.3 Attack types supported in SACRO-ML documentation

The SACRO-ML Attacks section enumerates the following attack types and supporting components:

- LiRA Attack
- Worst Case Attack
- Structural Attack
- Attribute Attack

as well as supporting artefacts such as “Target”, dataset handlers, metrics, reporting, and JSON outputs.

3.4 Researchers should be aware of following supporting components

The Attacks documentation references several building blocks that matter operationally:

- *Target*: an object that holds the model and relevant dataset references for attack evaluation
- *Dataset Handlers:* utilities to structure and manage datasets used for attack evaluation
- *Metrics:* summary measures produced from attack evaluation
- *Report + JSON outputs:* standardised outputs intended to support consistent review and auditability

4. How SACRO-ML is used in the SDE workflow

SACRO-ML is used as a structured approach to ML disclosure risk assessment:

1. **Ante-hoc (SafeModel)**

The researcher develops/trains the model using SafeModel wrappers and/or SafeModel checks, producing checkfile output and an export artefact suitable for review.

2. **Post-hoc (Attacks)**

Attack simulation evaluates the trained model under controlled conditions, producing risk evidence and reports to support output checker review.

4.1 Responsibilities (researcher vs output checker)

Researcher responsibilities (pre-submission)

Researchers are responsible for:

- Ensuring a defensible training/evaluation approach (including a valid train/test split where required for evaluation)
- Saving the model artefact in an agreed format
- Generating and retaining the checkfile / structured outputs as evidence for review
- Providing necessary contextual information for output checking (model purpose, model family, split approach, and evaluation outputs)

This aligns with SafeModel's design intent: to provide structured researcher feedback and a reviewable report artefact.

Minimum Submission Requirements

For a model to be assessed, the researcher must supply at minimum:

- Model file: model.pkl (sklearn/joblib), or equivalent for the framework in use.
- Data used for training and testing
    - train_X.npy
    - train_y.npy
    - test_X.npy
    - test_y.npy (or clearly named equivalents)
    - README / model card
- Brief description of:
    - Model name and purpose
        - e.g. "Case – Diabetes multifactor Random Forest"
    - Model type
        - e.g. RandomForestClassifier
- Features used
    - e.g. n_visits, n_conditions, n_drugs, n_measurements, n_procedures
- Outcome definition
    - e.g. has_diabetes
- Any important preprocessing steps
    - If any of these are missing, request for clarification

Please refer to the example notebook provided at the end for producing egress files.

Output checker responsibilities (formal assessment)

Output checkers are responsible for:

- Running standardised checks and simulated attacks as required by local governance
- Interpreting results in context (data sensitivity, cohort size, intended release, model type)
- Issuing a release decision or requesting remediation

SACRO-ML provides standard reports and JSON outputs to support consistent review.

4.2 Typical outcomes

After review, typical outcomes include:

- Approval to release the model artefact (possibly with conditions)
- Request to revise model configuration and retrain
- Request to adjust the planned release artefact (e.g., release summary metrics rather than the model), or non-approval where residual risk is not acceptable

5. Researcher interaction with SACRO-ML (including examples)

This section explains, in practical terms, how researchers can use SafeModel, and how researchers may optionally run attack simulations (where permitted) to identify potential issues earlier and reduce avoidable iteration during output checking. Please download the Example Handbook here.

5.1 What is expected vs optional

- **Expected:** use SafeModel (or equivalent local workflow) to generate structured artefacts for review, including a checkfile report and a saved model artefact suitable for controlled evaluation
- **Optional:** run simulated attacks as a *pre-submission diagnostic* to understand privacy risk signals. Formal decisions remain with output checkers

5.2 Example A: Using SafeModel during model development (illustrative workflow)

The SafeModel base class exposes a typical workflow comprising:

1. Train the model (fit)
2. Save the model artefact
3. Run a preliminary check (parameter safety checks)
4. Produce a structured submission artefact using request_release (which also generates a report for output checking)

Illustrative (documentation) sequence

- Train: fit(X, y)
- Save: save(name="model.pkl")
- Check: preliminary_check(...)
- Prepare for review: request_release(path=..., ext=..., target=...)

5.3 Example B: Running a simulated attack as a researcher pre-check (optional)

The SACRO-ML Attacks documentation lists the available attack types for post-hoc evaluation, including LiRA, Worst Case, Structural, and Attribute attacks.

Important conditions (documentation-grade guidance) :

- Attack simulation assumes the availability of appropriate test data not seen during training
- Attack simulation is not intended for certain instance-based methods or scenarios without a valid split

Please refer to the more comprehensive guide here https://sacro-tools.org/index.html

6. SACRO-ML – Getting Started (Researchers)

Purpose:

- Use **SafeModel** during model development
- Run built-in safety checks
- Prepare a model for disclosure review

This document focuses on **how to use SACRO-ML in practice**.

i. When Should You Use SafeModel?

Use SafeModel **after you have trained a model using your normal workflow**, and before requesting model egress.

Typical scenario:

1. Prepare dataset
2. Split into training and test sets
3. Train model (e.g. RandomForest, SVC, etc.)
4. Evaluate model performance
5. Wrap with SafeModel
6. Run checks
7. Decide whether to refine or submit

SafeModel adds structured disclosure checks.

ii. Prerequisites

Before using SafeModel, you should have:

- A trained machine learning model
- A valid train/test split

- Test data not used during training
- Model evaluation metrics (e.g. accuracy, AUC, etc.)

iii. Standard Researcher Workflow with SafeModel

Step 1 – Train Your Model (Normal Workflow)

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=20, # seleted for demo purposes
    random_state=42
)
model.fit(X_train, y_train)
```

Evaluate normally:

```
model.score(X_test, y_test)
```

Step 2 – Use SafeModel Wrapper

Replace your base model with a SafeModel equivalent.

Example:

```
from sacroml.safemodel.classifiers.saferandomforestclassifier
import SafeRandomForestClassifier
safe_model = SafeRandomForestClassifier(
    n_estimators=20,
    max_depth=8,
    min_samples_leaf=5,
    random_state=42
)
safe_model.fit(X_train, y_train)
```

This trains the model as usual but enables structured safety checks.

Step 3 – Run Preliminary Checks

```
safe_model.preliminary_check()
```

This checks for:

- Risk-prone parameter configurations
- Small leaf sizes
- Over-fitting indicators
- Structural disclosure signals

If warnings appear, you should:

- Adjust parameters
- Retrain
- Re-run checks

This may require several iterations.

iv. Decision Point

After training and running checks:

Is the model ready for disclosure submission?

Decision should be based on:

- Performance metrics
- SafeModel warnings
- Stability of evaluation results
- Intended release context

If not ready → refine model and retrain.

If ready → proceed to artefact generation.

v. Preparing for Disclosure Submission

Once satisfied with the model results:

```
safe_model.request_release(...)
```

This generates:

- A structured checkfile/report
- Model metadata
- A submission-ready artefact

You will then:

- Submit model artefact
- Provide brief description (purpose, features, outcome definition, preprocessing)
- Await disclosure review

vi. Optional: Running Simulated Attacks (Advanced)

Researchers may optionally run simulated attacks (e.g., LiRA) to assess empirical privacy risk before submission.

This is:

- Optional unless required
- Useful for high-risk or externally deployed models
- Intended as a structured diagnostic

vii. What Happens After Submission?

Output checkers may:

- Approve and release

- Approve with conditions
- Request revision
- Reject if residual risk is unacceptable

SafeModel helps reduce avoidable iterations by surfacing issues early.

viii. Full Documentation

For detailed explanations, see:

SACRO-ML Comprehensive Researcher Guide

Includes:

- Why ML models require disclosure risk assessment
- Membership inference risk
- Structural leakage
- Attribute inference
- Overfitting and memorisation
- Attack types (LiRA, Worst Case, Structural, Attribute)

Glossary

Ante-hoc Assessment – Evaluation of disclosure risk during model design and training stages. Focuses on parameter choices and modelling decisions before model release.

Attribute Inference Risk – Risk that sensitive attributes of individuals can be inferred from model outputs. Occurs when models strongly encode relationships between features and outcomes.

Checkfile – A structured report generated by SafeModel for review. Contains model metadata, warnings, and a recommendation for disclosure suitability.

Disclosure Risk – The possibility that released outputs reveal information about individuals. In ML, this arises from model behaviour rather than direct data exposure.

LiRA Attack –Likelihood Ratio Attack used to detect membership inference risk. Compares model behaviour on training vs non-training data points.

Membership Inference Risk – Risk that an attacker can determine if a record was used in model training. Often linked to overfitting and differences between train and test performance.

Model Artefact – The trained model file or pipeline intended for release (e.g., .pkl). Considered an output that must undergo disclosure control before egress.

Overfitting – When a model learns training data too closely and fails to generalise. Increases disclosure risk by enabling memorisation of individual records.

Post-hoc Assessment – Evaluation of disclosure risk after model training is complete. Typically involves simulated attacks to test model behaviour under risk scenarios.

SafeModel – A wrapper-based approach for ML models that enables structured risk checks. Provides warnings, validation, and generates artefacts for disclosure review.

Simulated Attacks – Controlled techniques used to estimate privacy risks in trained models. Used to test whether models leak information about training data.

Structural Leakage – Disclosure risk arising from the model structure itself (e.g., small tree leaves). Often linked to parameter settings that allow memorisation.

Train/Test Split – Division of data into training and unseen test sets for evaluation. Essential for assessing generalisation and enabling attack-based risk testing.

Worst Case Attack – A membership inference attack estimating the maximum possible risk. Provides an upper bound on how much information the model may leak.